Foundations of Software Development, Data Processing, and Visualization

Software / Machine Learning Domain

Submitted By

Snigdha Routray

Registration Number: 240962442

Repository

GitHub Repository:

https://github.com/SnigdhaRoutray/Synergy_TP

Submission Date

27 June 2026

Manipal Institute of Technology

Abstract

In this report we will discuss the ideas behind the work we did in the five tasks of the Software/ML Task phase. The tasks include creating a Python coding environment which can be reused, writing code in a modular way, working with structured, tabular data, cleaning up messy datasets, and visualizing data. This report does not discuss the code used but rather it discusses the concepts used and their importance, the reason for which the design was chosen, and the lessons learned from turning the unstructured data into structured one via the pipeline process.

1. Introduction

The tasks build on each other progressively. Task 1 sets up the clean and repeatable project environment. Task 2 provides knowledge about modular coding in Python with realistic file handling operations. Tasks 3 and 4 are concerned with the representation and cleansing of tabular data. Task 5 involves representing the cleansed data visually. Task 6 is this report, linking theory to practice. The overall workflow followed throughout the task phase is illustrated in Figure 1, showing the progression from raw data to meaningful visualizations.

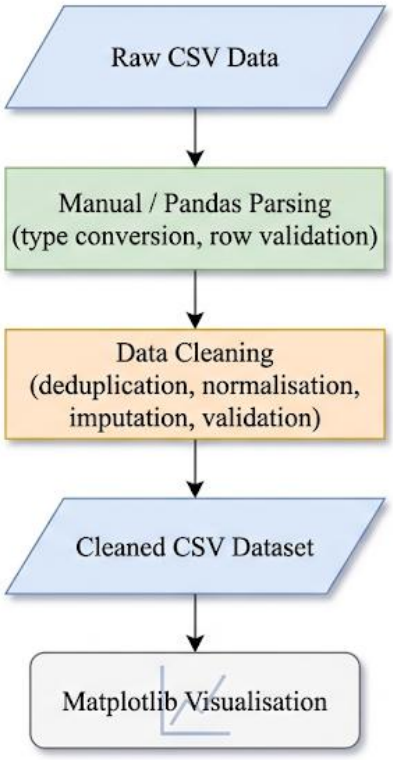

Figure 1. Workflow illustrating the transformation of raw CSV data into a cleaned dataset and its subsequent visualization using Matplotlib.

2. Development Environment and Version Control

2.1 Git and GitHub

Git is a distribution control system which records all the changes that have been made to a project. GitHub is a website where such changes can easily be viewed and shared. The concepts which were applied during the process of performing the tasks are:

- Repository: The term "repository" describes the folder which contains the source files of a certain project.
- Commit: It is the point of time when all the changes within the repository are saved.
- Branch: This is a separate line of development.
- Push/Pull: This is the transfer of commits from the local machine to the remote repository on GitHub.

Maintaining a repository requires having a README file, correct folder organization, and a .gitignore file.

2.2 Virtual Environment and Dependencies Isolation

Python virtual environment is defined as the creation of an isolated version of the Python interpreter and its libraries. In absence of this, all the projects in a system will have the same versions of all the installed packages. Different projects may need different versions of the same package. Using python3 -m venv creates the isolated environment. Once the required packages have been installed, use of command pip freeze > requirements.txt saves the exact version of all the installed packages. Any user that clones this project and uses pip install -r requirements.txt gets the same environment. The .gitignore file contains the list of files and folders that should not be tracked by Git. This is because the virtual environment folder is large and platform specific and can be replicated using requirements.txt file. Cache folders for Python, namely __pycache__ and .pyc are also added in gitignore for the same reason.

2.3 Linux Command-Line Basics

In Linux, file system access can be done directly using the command-line interface. The commands used in Task 1 fall into the following categories:

- Navigation and checking: pwd (print working directory), ls / ls -la (show files, including the hidden ones), cd (change directory)
- Files and directories manipulation: mkdir (create directory), touch (create an empty file), cp (copy), mv (move/renaming), rm (deleting)

- Content manipulation: cat (displaying file content), echo (displaying text on screen), head/tail (displaying first/last lines from a file), grep (searching for strings in a file), find (find a file), wc (displays number of lines/words/characters in the file)
- Permissions: chmod changes the ability to read/write/execute the file

3. Python and File Handling Concepts

3.1 Function Definition and Type Hints

Separating the program into functions improves readability, testability, and reusability. All functions in Tasks 2 to 5 perform a single task each Putting them together into a single chunk of code will make testing and modification hard. The type hinting does not influence the run time behaviour of a function; however, it describes what kind of input and output are expected from that function.

3.2 Lists and Dictionaries

List is an example of a mutable data structure that maintains order. According to Task 3, each line from the CSV file is read as a string, followed by splitting through commas, and converting into a dictionary with key value pairs. It is more reliable to represent the lines as dictionaries ({‘name’: ‘Aarav’, ‘score’: 8, ‘submitted’: True}) rather than by index since each element will be clearly named.

3.3 File I/O and Context Manager

Using open() to read/write from/to file inside with statement ensures that file handler is closed automatically in the end irrespective of exception being raised or not. To print JSON-formatted output, json.dump(data, file, indent=4) will generate human-readable JSON data that is easily viewable and can be tracked.

3.4 Exception Handling

Real world situations involve instances where the input is not well formed. The second exercise involves the situation where the input contains errors. This is resolved using the following exception classes: FileNotFoundError (the input file does not exist), pd.errors.EmptyDataError (the input file is empty), and ValueError (a non-numeric score in the input file).

4. CSV Parsing and Pandas

4.1 Format of CSV files

CSV files are simple text files. The first line consists of names of columns while other lines represent rows in which values are separated by commas. The problems associated with the parsing of CSV file are as follows:

- Type conversion: Every field in CSV file is considered as string type.
- Submitted normalisation: Strings like "yes", "Yes", "Y", "1" and "true" are the same strings but they can be different.
- Malformed rows: Rows differing in number of fields in comparison to the header should be skipped.

4.2 Manual Parsing vs. Pandas

The use of manual parsing in Task 3 before working with pandas is used to show what pandas does. Using manual parsing with open() and str.split(",") illustrates each assumption that pandas makes on its own. The pandas' read_csv() method handles all of those mentioned above, including missing value handling, type inferencing, encoding detection, etc. in one fell swoop. It's easier to write and less likely to make mistakes when used on clean data. However, the downside is that mistakes in dirty data can be masked by the default parameters in pandas. Task 3 proves that the results obtained via the two different methods are equal for clean data.

5. Data Cleaning

5.1 Why Raw Data Is Rarely Clean

The raw data set in task 4 included all possible types of real-life data quality problems:

- Duplicate records: S005 had two similar entries.
- Incorrect category names: "ml", "ML", "MACHINE LEARNING", "web", "Web Dev" and "web development" and "electronics" were all listed in the domain category while representing only four different domains.
- Units mismatch: height was entered as "170 cm" and "1.62 m"; "65 kg" and "55kg" (with and without spaces) were used for weight.
- Words used to represent numbers: The words "nine" and "two" appeared in columns that must only accept numeric values.
- OutOfRange values for attendance: -10 and 105 are out of range [0-100].
- Missing values: several scores and attendance cells were empty.

5.2 Cleaning Decisions

Data cleaning process needs decision-making and keeping the record about these decisions so as not to make any random changes to the data. All decisions made are noted down in cleaning_report.md:

- Duplicate entries have been removed using drop_duplicates() function.
- The domain names have been cleaned using a dictionary that contains canonical form of all versions of domain name (e.g., ML). Case insensitive comparison was used as the first step (.str.lower() function) and only then the lookup in the dictionary.
- The attendance value which lies out of 0-100 range was replaced with the median value of all valid values rather than deleted since deleting the row means deleting all information about the student.
- Numbers represented in words form were converted using word2number library.
- Height in metres was identified through "m" and multiplied by 100 to convert it to cm

5.3 Validation After Cleaning

Validation verifies that the data after change is accurate. The validator in Task 4 tests for:

- There are no duplicates of student_id.
- attendance_percent is numeric and lies between 0 and 100.
- domain has only four values.
- submitted has only two values: True/False.
- There are no null values in any required column.

6. Visualization

6.1 Purpose of Visualisation

Numbers in the table convey facts. Plots convey patterns. The bar chart of the average score per domain answers the question "which domain is performing best" immediately. Visualization is not just about aesthetics. It translates information into a format that can be consumed easily by the human visual system.

6.2 The Three Plots and Their Message

- **Average Score Per Domain (Bar Chart):** The bars represent domains (ML, Web, Electronics, Mechanical), and their heights reflect the average score of all students in each domain. This chart answers if there are differences in performance depending on the technical field and to what extent.

- **Attendance vs. Score (Scatter Plot):** The scatter plot shows a point per student, with the x coordinate representing the percentage of attendance and the y coordinate showing the student's score. If attendance is positively correlated with good results, we will see a rough diagonal.
- **Status of Submission (Bar Chart):** There are two bars representing the number of students who have submitted and those who have not submitted respectively.

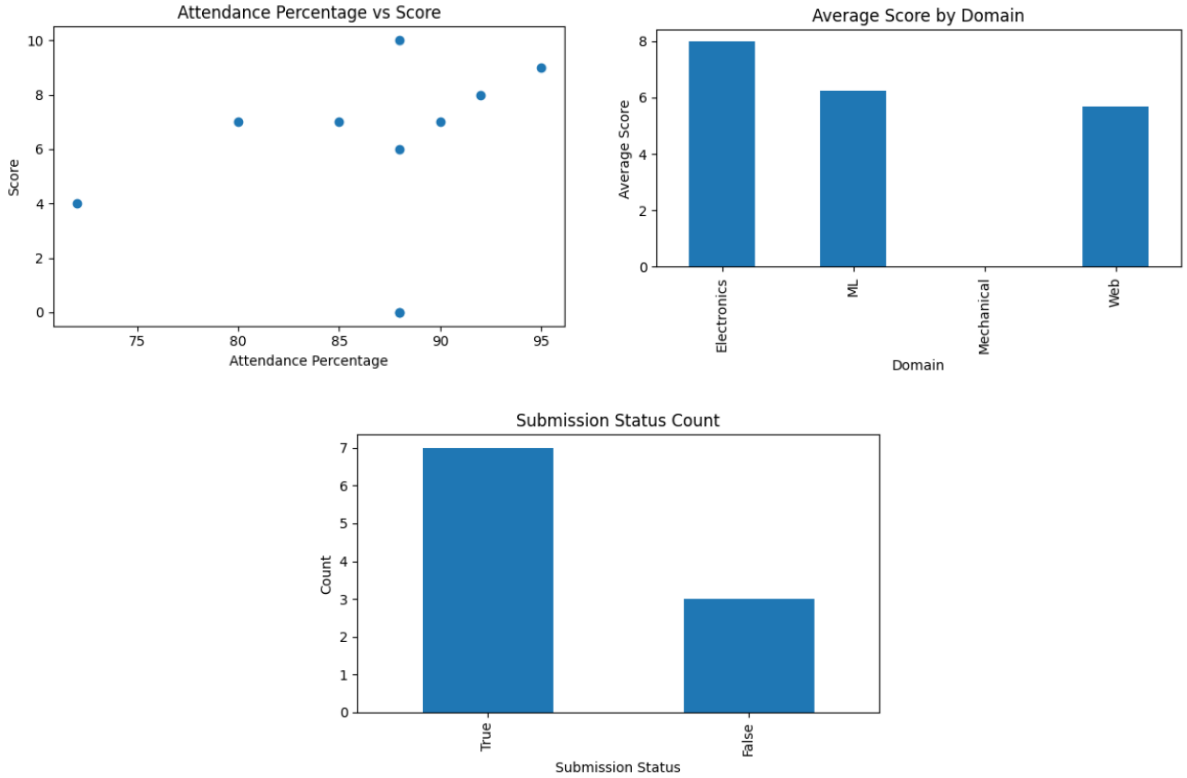

Figure 2. Generated Visualizations

7. Conclusion

Tasks 1 through 5 create a full data pipeline from start to finish:

- Environment setup guarantees reproducibility and portability of the project.
- Modular Python coding allows for separation of concerns and testability.
- CSV Parsing: First, manually, then using pandas, we gain insight into the assumptions underpinning high-level libraries.
- Data cleaning turns real-life dirty, inconsistent data into a solid analysis-ready data set.
- Visualization turns clean data into insight.

These ideas altogether constitute a solid basis for further development of machine learning and data engineering projects.

Table 1. Summary of Tasks 1-5

Task	Title	Concept
1	GitHub, Virtual Environment, and Linux Basics	Project setup, version control, dependency isolation, command-line usage
2	Python Recap	Functions, type hints, file I/O, exception handling, dictionaries
3	Manual CSV Parser and Pandas Comparison	CSV structure, manual parsing, pandas loading, type conversion
4	Messy CSV Cleaning	Missing values, duplicates, unit normalisation, validation
5	Matplotlib Visualisation	Bar charts, scatter plots, matplotlib API, non-interactive figure saving

References

[1] Git, "Git - Documentation," Git-scm.com. https://git-scm.com/doc

[2] Python Packaging Authority, "Installing packages using pip and virtual environments," Python Packaging User Guide. https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/

[3] The pandas development team, "pandas documentation," pandas.pydata.org. https://pandas.pydata.org/docs/

[4] The Matplotlib development team, "Matplotlib documentation," Matplotlib.org. https://matplotlib.org/stable/contents.html

[5] Python Software Foundation, "json — JSON encoder and decoder," Python 3 Documentation. https://docs.python.org/3/library/json.html

[6] A. Gupta, "word2number," Python Package Index (PyPI). https://pypi.org/project/word2number/

[7] Python Software Foundation, "csv — CSV File Reading and Writing," Python 3 Documentation. https://docs.python.org/3/library/csv.html